

① 1.1)

SQL: estructuración del lenguaje (SQL)
 indep físico/lógico
 fundamentos teóricos
 ACID

3V: Variación, Volumen, Velocidad

NoSQL

Escalabilidad
 Disponibilidad, Replicación, consistencia eventual

Performance en acceso a datos

BASE vs ACID

Schemelers/Flexible Scheme

SQL → "slow, expensive"

NoSQL → "efficient, cheap"

Feature	SQL	NoSQL
Data Model	Structured (table, row)	Flexible (JSON, KV, ...)
Schema	Fixed schema	Schema-less
Scalability	Vertical (scale-up)	Horizontal (scale-out)
Transactions	STRONG ACID compliance	eventual consistency
Query lang	SQL (standard)	not standard
Best for	Structured, relational data	unstructured data

SQL performance ↓ for large data volumes

→ NoSQL low latency for large data

∴ Better for read side.

SQL is still better at

- complex joins (Relational)
- multi record txs
- Consistency

- 1.2) • KEY-VALUE: $\neq$ columnas, significados de valor $\oplus$ numéricos de la clave.
- DOCUMENT: datos $\oplus$ documentos estructurados. Doc $\oplus$ unidad mínima de info, puede ser XML, JSON, ... Puede ser anidado
- WIDE COLUMN: tablas c/ columnas dimensionales que se pueden agrupar en familias de columnas. Simple se deciden x rowid.
- GRAPH DATABASE: Entidades son nodos y relaciones son ejes con propiedades, ideal p/ interconexiones complejas de datos

1.3) Namespace en KV es organización lógica a través de los nodos de las claves (no físicas) que evitan conflictos de colisión, y mejora la escalabilidad

1.4) Cuando orientamos nuestros en proceso la compra. x c/ key de usuario se mantiene el TTL de "sesión".

1.5) Comparar KV c/ Document, multiples, desarrollos.

Feature	KV	Document
Schema	Fixed schema	Flexible schema
Query	by exact keywords	Rich querying
RW	fast & simple lookup	slightly slower & trickier
Best for	simple, atomic operations & relations	supports more complex effects & relations
	Easy to implement	Better structure

1.6) Schema = structure that defines

- organization of data
- data types
- relationships (between table & collection)
- constraints.

in SQL, defined by DDL
 CREATE TABLE, ALTER TABLE, etc

Schema-less Document DB

- Pros
 - flexibility → each doc $\neq$ fields
 - faster development → no schema modification or compliance
 - easy handling of heterogeneous data
- Cons
 - ↓ consistency / violation (more unenforced data)
 - ↓ data integrity (hard to enforce relationship types & constraints)
 - ↑ Query Complexity
 - Debugging & maintenance

1.7) Demanding: p/ ↓ latencia en búsqueda de info.

1.8) ColumnFamily Databases vs other NoSQL

C FDB → each row can have $\neq$ number type of columns

- they are grouped into families
- data is stored by column, not by row

- good for writing throughput,
- read speed: KV > C FDB > Document
- Rich querying
- good for fbt, time-ordered data

1.9) Eventual consistency

→ eventualmente los nodos de la DB envían todos c/ el último valor (no actualizado)

⇒ ↑ Availability (responde sin hallar)

pero quizás c/ valor viejo

A medida que $t \rightarrow \infty$, los $\neq$ clientes/lectores ven los escrituras.

1.10) Teorema de CAP:

solo se pueden 2 de 3 prop p/ minimo comprometidos

- Consistency: todos ven mismo dato al mismo tiempo
- Availability: si $\exists$ conexión c/ un nodo → puede R/W
- Partition tolerance: Cliente puede sobrevivir a rotura de comunicación que los divide en particiones que no pueden comunicarse entre ellos.

Derivado → si necesitas tolerar particiones y al mismo tiempo alta de manera inesperada

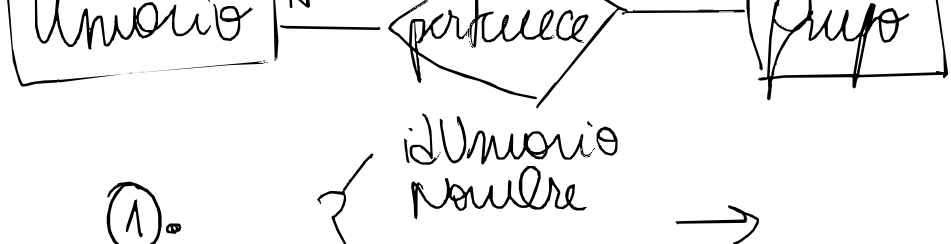
⇒ solo se puede PC o PA

Excepción de CAP: PACELC

→ CAP $\oplus$ Else → latency (response time)

→ consistency

} →



①. { idUnorio nombre →

②. { }

③. ← { }

①. grupo: {

▷ props grupo

unorio que pertenece { unorio: {

▷ props unorio

}

• unorio: {

▷ props unorio

▷ pertenece a grupo: [groups id]

}

2.2)

a) Normalización de los datos, para poder hacer una referencia de editoriales y no una incrementación.

Bonitamente por de

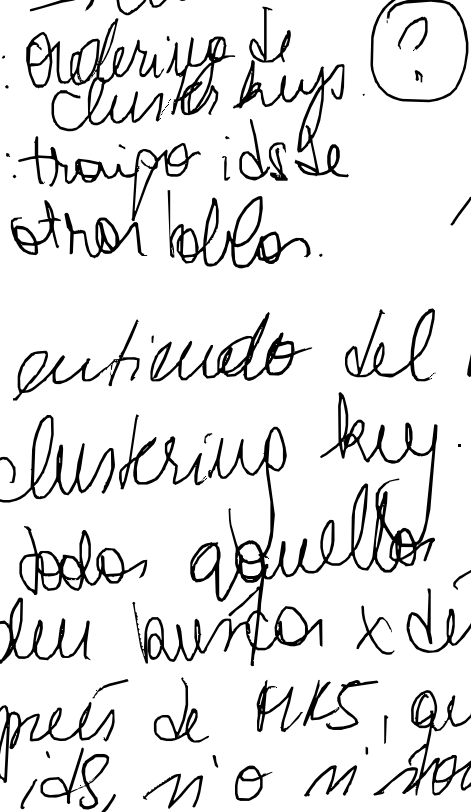
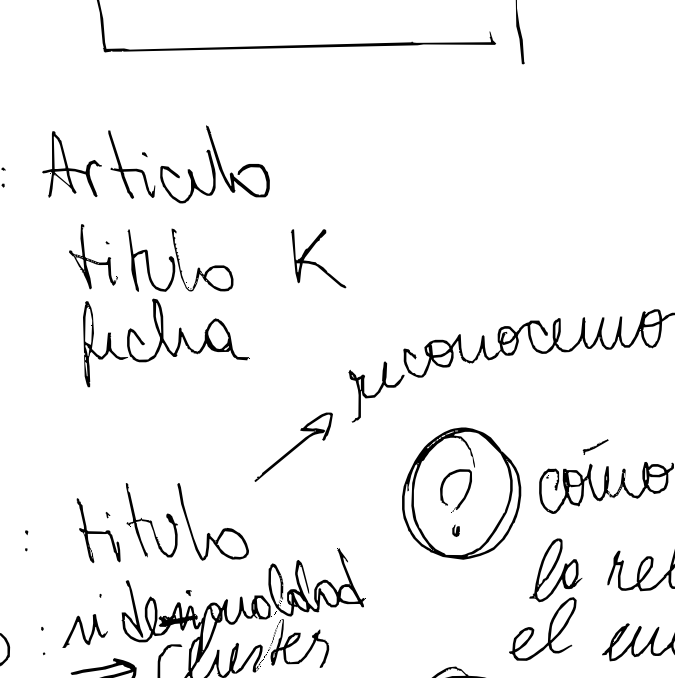
} a →

b) Cuando siempre que necesite el libro junto c/lo dato de su editorial.

Y con referencia cuando no es necesario (no se necesita la función como el unorio)

2.3)

a)



MR1: Articulo

Titulo K

Fecha

reconocemos keys

MR2: titulo

MR3: no desigualdad

MR4: ordenamiento de clusters keys

MR5: traigo id de otros libros.

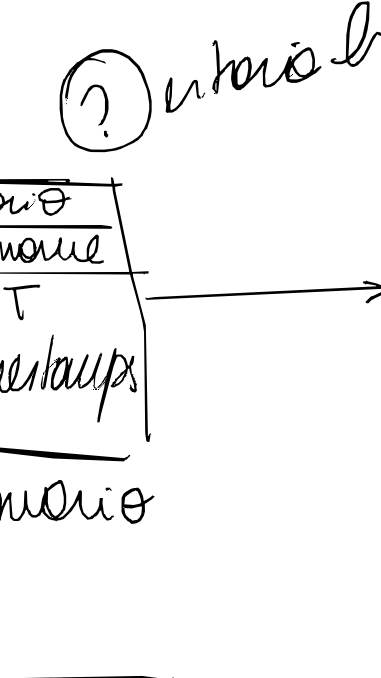
¿cómo reflejamos la relación en el modelo?

¿MR2 reconoce K y C o solo K?

¿no entiendo del todo los clustering keys. Son todos aquellos que pueden hacer x desigualdad?

Después de MR5, que traemos los ids, si o si son cluster keys?

b)



MR1: Autores, Articulos

Fecha K

nombre C↑

apellido C↑

Titulo C↑

¿si buscamos x fecha, el titulo opera como K? si opera como K, aparece en MR1 o MR5?

Si aparece como C, aparece en MR5, no?

MR2: fecha

MR3: desigualdad? NO

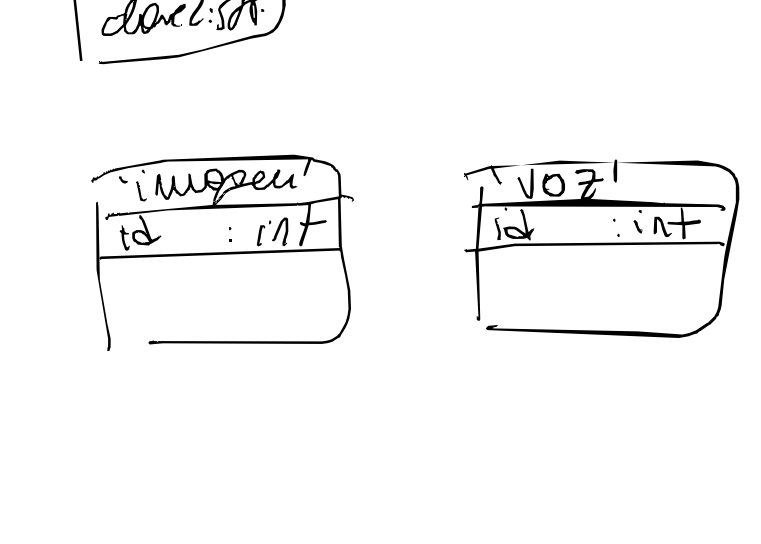
MR4: no pide ordenamiento.

MR5: nombre, apellido non C↑

Titulo C↑

2.4)

¿unorio bien?



2.5)

a)



USER

DC

DG

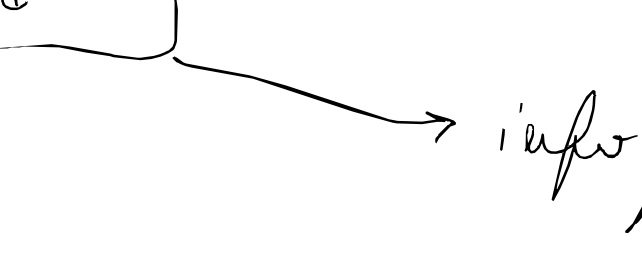
USER

DC

DG

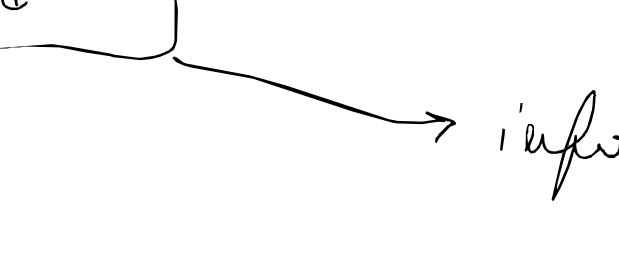
¿cómo lo RESOLVERIAN? si luego esto que obido de la información puede?

b) ¿Lo hacemos todo en tablas separadas?

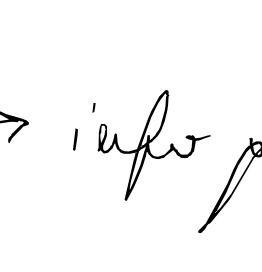
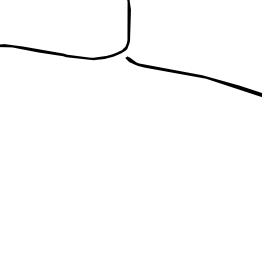


o le damos una relación?

c)



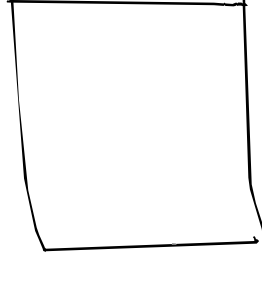
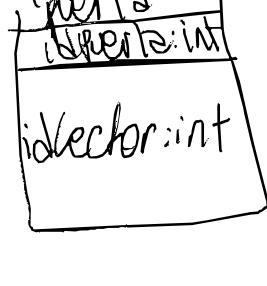
d)



info chicos

info chicas

2.6)



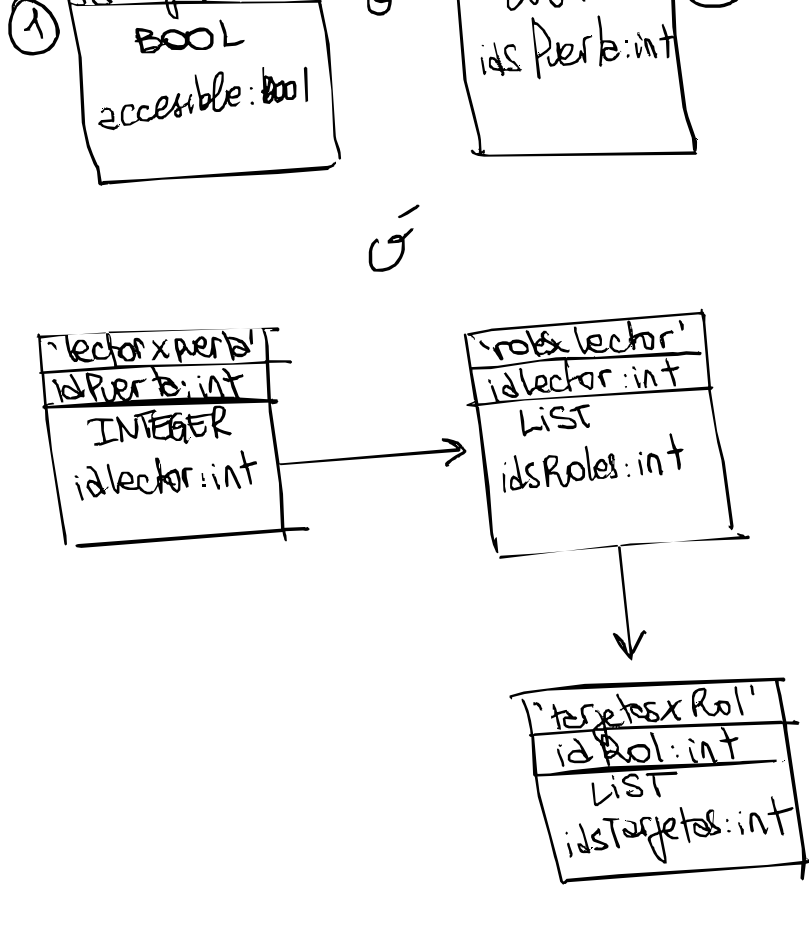
tarjeta $\rightarrow$ identificador
 lista x lectores
 puerta $\rightarrow$ id: numero
 tiene lector
 tiene permiso?
 $\rightarrow$ puerta recibe
 roles = grupo de identificadores
 id tarjeta puede $\in$ a $\neq$ roles

tarjeta $\rightarrow$ puertas que poseo
 puertas que no poseo
 momento en que intento abrir puerta x
 puede ocurrir a puerta?

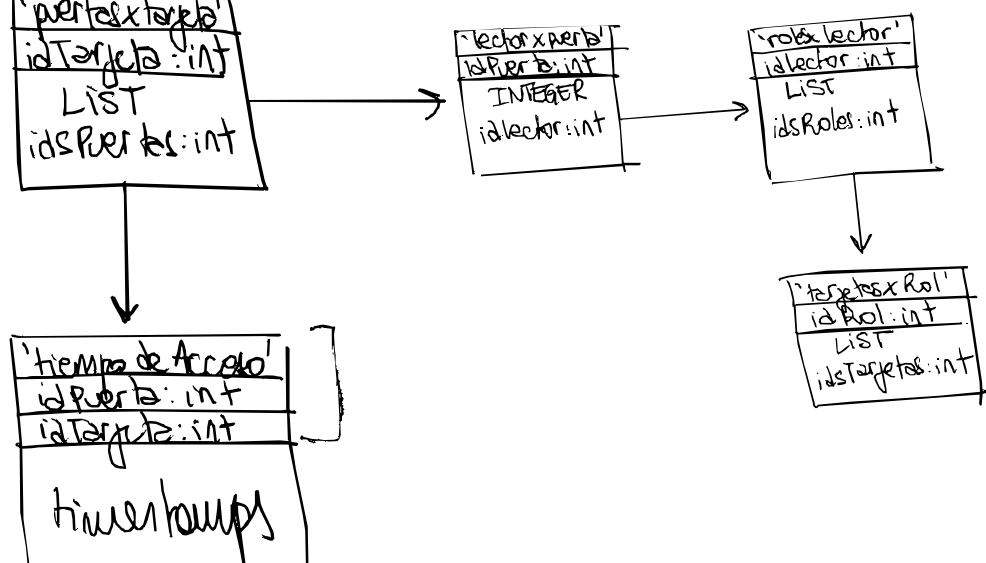
a) Puede ocurrir que desactivar la tarjeta es eliminable de la DB?
 (?) PUEDE USAR LO QUE HAY EN EL HASH? cuando que



SE ME OCURREN 3 MANERAS P/DET SI TARJETA PUEDE ACCEDER A PUERTA

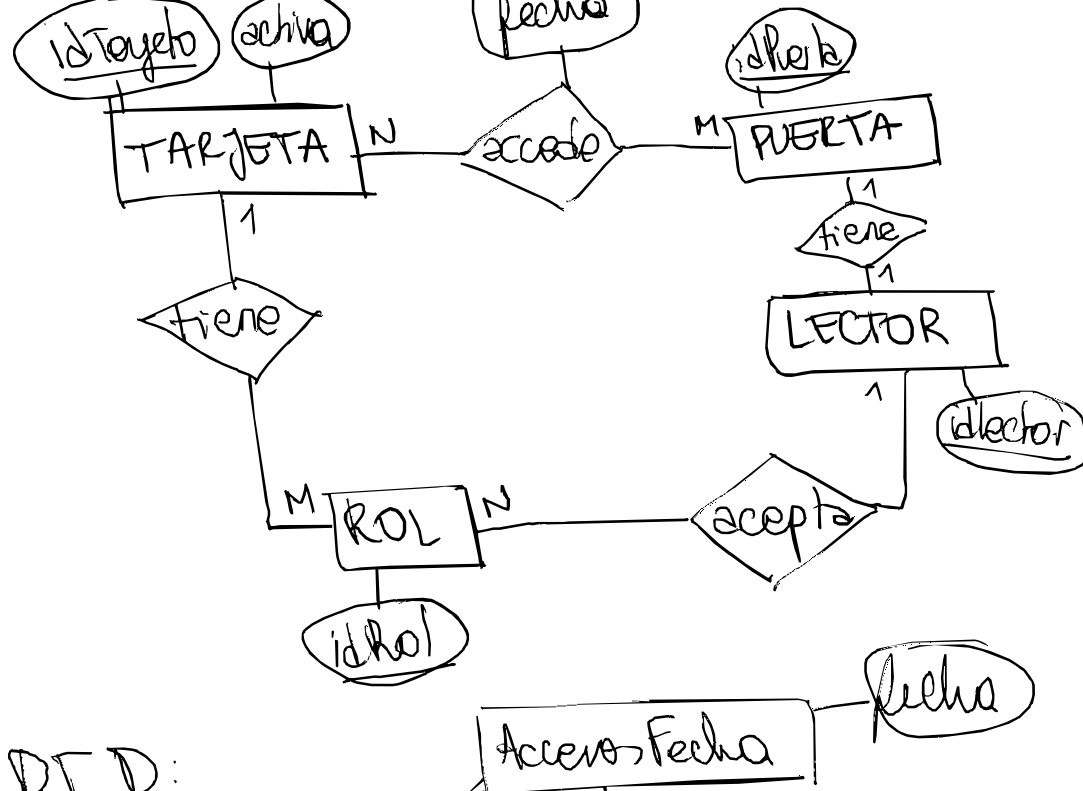


TAMBIEN PODRIA HACER ALGO ASI:

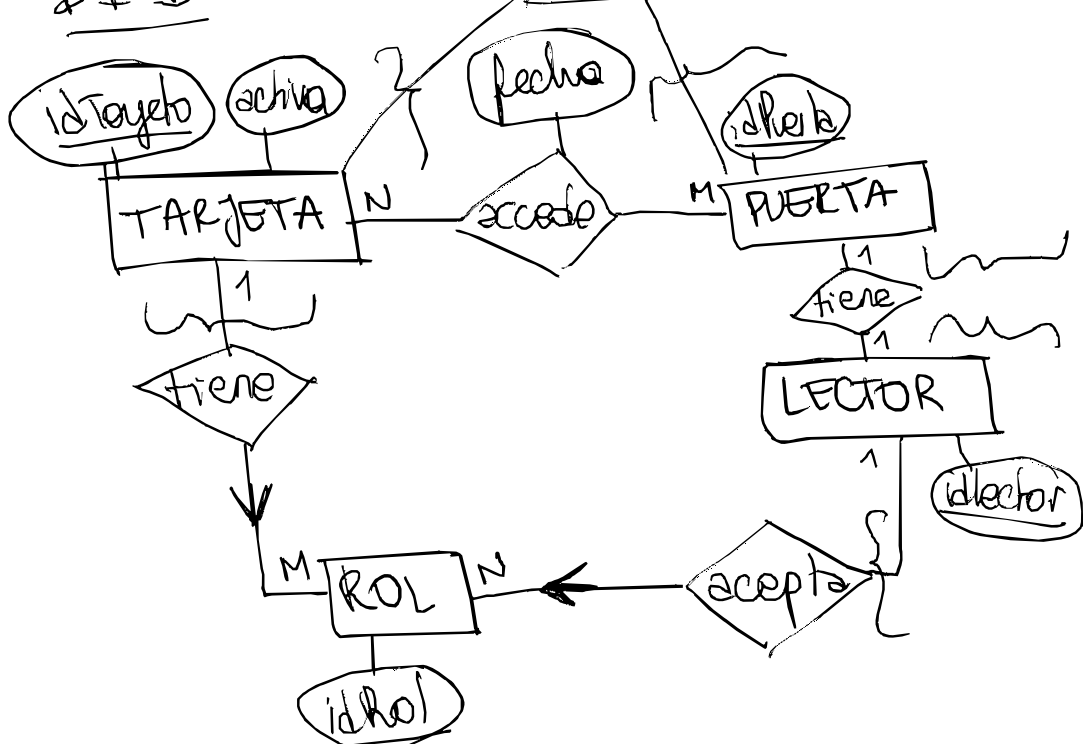


NO SE HONESTAMENTE QUE ES MEJOR me parece a que la 2^a es mas SQL.

b) DER:

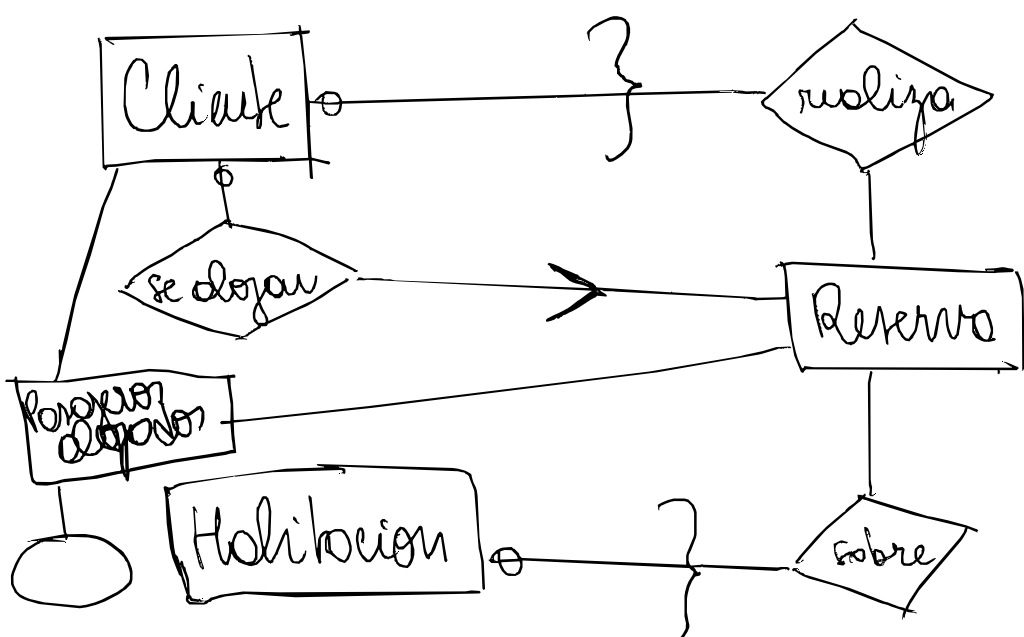


DI D:



creo que en este caso habria bien mas un documento auxiliar xq en b) dicen que puede crecer mucho

2.7)



Cliente reserva en rango de fechas
 ↳ 1 habitación → precio x noche
 ↳ p / # personas (cambio x tiempo)

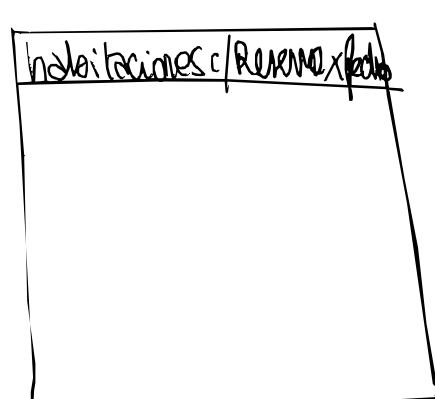
Consulta reserva → detalle habitación
 ⊕
 datos cliente
 ⊕
 links paginas

Ⓢ como manejar la lista?
 doc auxiliar

JSON Schema:

```
"Reserva": { "type": "object",
  "properties": {
    idReserva: { type: string },
    FechaDesde: { type: string,
      format: date-time
    },
    FechaHasta: { type: string,
      format: date-time
    },
    clientesReserva: { type: object,
      properties: { ... }
    },
    habitacionReserva: { " " }
  }
}
```

b)



MR1: Reserva, Habitación

fechaDesde k
 ↗ nro C↑
 precio C↓
 categoria
 idReserva C↑

MR2: fechaDesde

MR3: disponibilidad? NO

MR4: precio C↓

MR5: nro, idReserva

c)

